

C++ 底层例题代码集（17 例）

18 个文件 · 979 行 · 生成于 2026-10-09 14:55 · 行号对应源文件实际行号

05-例题代码索引.md

Markdown

C++ 底层例题代码集 · 目录

17 个例题，每个文件从新的一页开始；讲解见 01/02/03-第X阶段-例题讲解.md。 对照用法：先看讲解里的"真实输出"，再翻到这里的代码页，边读边在纸上标出关键行。

第一阶段 · 内存地基（第 1-2 周）

页	文件	主题
2	p1_01_sizeof.cpp	空类、基础类型、对齐、vptr 的字节代价
4	p1_02_layout.cpp	成员顺序与填充、offsetof、逐字节 dump
8	p1_03_stack_heap.cpp	栈向下增长、堆分配、作用域与地址复用
12	p1_04_vptr.cpp	vptr 位置、虚表内容、绕过语法直接按表调用
13	p1_05_polymorphism.cpp	虚函数 vs 非虚函数、两张虚表、虚析构

第二阶段 · 容器与所有权（第 3-6 周）

页	文件	主题
14	p2_01_string.cpp	SSO 边界、容量跳变、reserve 的效果
15	p2_02_vector.cpp	扩容序列、移动而非拷贝、迭代器失效
16	p2_03_raii.cpp	逆序析构、异常路径、禁止拷贝只允许移动
18	p2_04_unique_ptr.cpp	所有权转移、传参三态、自定义删除器
19	p2_05_shared_ptr.cpp	引用计数、循环引用泄漏、weak_ptr 破坏

第三阶段 · 边界与排坑（第 7-12 周）

页	文件	主题
20	p3_01_multi_inherit.cpp	两个 vptr、指针偏移、三种 cast 的区别
21	p3_02_virtual_inherit.cpp	菱形继承两份数据、虚继承的 vbptr
22	p3_03_overflow.cpp	堆越界写 (ASan: heap-buffer-overflow)
23	p3_04_use_after_free.cpp	释放后使用 (ASan: heap-use-after-free)
24	p3_05_double_free.cpp	重复释放 (ASan: attempting double-free)
25	p3_06_project.cpp	综合项目：多态 + 容器 + 智能指针 + 文件
27	p3_07_diagnose.cpp	排坑四步法：切片、悬空引用、容器扩容

三个"故意犯错"的例子请用 clang++ -std=c++17 -g -O0 -fsanitize=address 编译， macOS 上不要加 detect_leaks=1（会报 not supported），用 ASAN_OPTIONS=abort_on_error=0 运行。

```

1 // 例题 1-1: 空类、基础类型、对齐、vptr 到底各占多少字节
2 // 观察点: sizeof 是"类型合同", 不是"实际用了多少内存"
3 #include <cstdint> //这个库提供了 size_t 类型
4 #include <cstdio> //这个库提供了 printf 函数
5
6 struct Empty {}; // 空类: 编译器必须给 1 字节占位
7 struct OneChar {
8     char c;
9 };
10 struct CharInt {
11     char c;
12     int i;
13 }; // 1 + 3 填充 + 4 = 8
14 struct IntChar {
15     int i;
16     char c;
17 }; // 4 + 1 + 3 尾部填充 = 8
18 struct HasFunction {
19     void f();
20 }; // 非虚成员函数不占对象内存
21 struct HasVirtual {
22     virtual void f();
23 };
24
25 int main() {
26     printf("sizeof(Empty)      = %zu\n",
27           sizeof(Empty)); // z: 表示参数类型是 size_t
28     printf("sizeof(OneChar)    = %zu\n",
29           sizeof(OneChar)); // u: 表示按无符号十进制整数输出
30     printf("sizeof(CharInt)    = %zu   (char 后补 3 字节)\n",
31           sizeof(CharInt));
32     printf("sizeof(IntChar)     = %zu   (尾部再补 3 字节)\n", sizeof(IntChar));
33     printf("sizeof(HasFunction) = %zu   (成员函数不占对象内存)\n",
34           sizeof(HasFunction));
35     printf("sizeof(HasVirtual)  = %zu   (只有一个 vptr)\n",
36           sizeof(HasVirtual));
37     printf("\n-- 基础类型 (%zu 位平台) --\n", sizeof(void *) * 8);
38     printf("bool=%zu char=%zu short=%zu int=%zu long=%zu long long=%zu\n",
39           sizeof(bool), sizeof(char), sizeof(short), sizeof(int), sizeof(long),
40           sizeof(long long));
41     printf("float=%zu double=%zu long double=%zu\n", sizeof(float),
42           sizeof(double), sizeof(long double));
43     printf("int*=%zu void*=%zu 函数指针=%zu\n", sizeof(int *), sizeof(void *),
44           sizeof(void (*)()));
45     Empty a, b;
46     printf("\n两个空类对象: %p 与 %p, 相差 %ld 字节\n", (void *)&a, (void *)&b,
47           (long)((char *)&b - (char *)&a));
48     return 0;
49 }

```

p1_01_sizeof.cpp • 内存结构图

按本机实测绘制 (Apple clang 2.1 / arm64 / macOS) : 1 格 = 1 字节, 数据成员、填充、vptr 分色。

数据成员 对齐填充 vptr (虚表指针) 占位字节

Empty 空类: 为什么不是 0 字节 sizeof = 1 字节



C++ 要求每个对象都有唯一地址, 空类也必须占 1 字节。

OneChar 只含一个 char sizeof = 1 字节



char 对齐 = 1, 所以大小就是 1, 没有任何填充。

CharInt char 在前, int 在后 sizeof = 8 字节



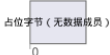
int 必须落在 4 的倍数上: char 占偏移 0, int 只能从偏移 4 开始, 中间 3 字节变成填充; 整体按 4 对齐。

IntChar int 在前, char 在后 sizeof = 8 字节



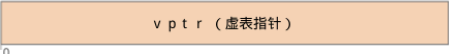
int 在偏移 0, char 在偏移 4; 整体必须按 4 对齐, 尾部又补 3 字节 —— 大小是 8, 不是 5。

HasFunction 只有非虚成员函数 sizeof = 1 字节



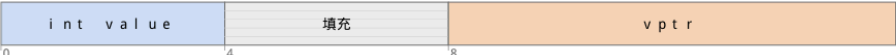
成员函数不占对象内存: 它编译成普通函数, 调用时额外把 this 传进去, 所以 sizeof 仍是 1。

HasVirtual 有一个虚函数 sizeof = 8 字节



出现虚函数后对象里多出一个 vptr (64 位下 8 字节), 它指向该类唯一的虚表。

WithVirtual int + 两个虚函数 (例题 1-4) sizeof = 16 字节



数据成员先按对齐落位 (int 在 0, 补到偏移 8), 再放 vptr —— 16 字节。

LooseLayout char -> double -> int (例题 1-2) sizeof = 24 字节



double 要 8 对齐, char 后面跳 7 字节; int 之后整体再补到 8 的倍数 —— 24 字节。

TightLayout double -> int -> char (例题 1-2) sizeof = 16 字节



同样的三个成员, 只改了顺序: 16 字节, 比上面省 8 字节 —— 成员排序也是优化。

- 三条规则
1. 成员按自身对齐要求落位: char = 1、int = 4、double = 8、指针 = 8; 放不下就往右挪, 空洞变成填充。

2. 结构体整体按最大成员对齐: 尾部常要补一段, 保证数组里每个元素都对齐 (IntChar 是 8 的原因)。

3. 空类占 1 字节占位; 有虚函数多加 8 字节 vptr —— 这两条与「有几个成员」无关。

```

1 // 例题 1-2: 成员顺序决定内存布局; offsetof 与字节 dump
2 // 观察点: 对齐填充出现在哪里、为什么调整顺序能省内存
3 #include <stddef>
4 #include <stdio>
5 #include <string>
6
7 struct LooseLayout { // 顺序不好: char → double → int
8     char c;
9     double d;
10    int i;
11 };
12
13 struct TightLayout { // 顺序调整后能省 8 字节
14     double d;
15     int i;
16     char c;
17 };
18
19 struct WithString { // 含库类型: std::string 在 libc++ 里是 24 字节
20     int i;
21     std::string s;
22     char c;
23 };
24
25 template <typename T> static void dump_bytes(const T &obj, const char *name) {
26     const unsigned char *p = reinterpret_cast<const unsigned char *>(&obj);
27     printf("%-12s (%2zu 字节): ", name, sizeof(T));
28     for (size_t i = 0; i < sizeof(T); ++i)
29         printf("%02x ", p[i]);
30     printf("\n");
31 }
32
33 int main() {
34     printf("sizeof(LooseLayout) = %zu\n", sizeof(LooseLayout));
35     printf("  offsetof c=%zu d=%zu i=%zu\n", offsetof(LooseLayout, c),
36           offsetof(LooseLayout, d), offsetof(LooseLayout, i));
37     printf("sizeof(TightLayout) = %zu\n", sizeof(TightLayout));
38     printf("  offsetof d=%zu i=%zu c=%zu\n", offsetof(TightLayout, d),
39           offsetof(TightLayout, i), offsetof(TightLayout, c));
40     printf("同样的三个成员, 顺序不同差了 %zu 字节。\\n\\n",
41           sizeof(LooseLayout) - sizeof(TightLayout));
42
43     LooseLayout loose{};
44     loose.c = 0xAA;
45     loose.d = 1.0;
46     loose.i = 0x11223344;
47     dump_bytes(loose, "LooseLayout");
48     printf(" → 0xAA 后面的 7 个 0 是填充, int 落在偏移 16。\\n\\n");
49
50     TightLayout tight{};
51     tight.d = 1.0;
52     tight.i = 0x11223344;
53     tight.c = 0xAA;
54     dump_bytes(tight, "TightLayout");
55     printf(" → 填充被挤到尾部, 只剩 3 字节。\\n\\n");
56
57     printf("sizeof(std::string) = %zu (libc++ 固定 24 字节) \\n",
58           sizeof(std::string));
59     printf("sizeof(WithString) = %zu  offsetof(s)=%zu  offsetof(c)=%zu\\n",
60           sizeof(WithString), offsetof(WithString, s),
61           offsetof(WithString, c));
62     WithString ws{};
63     ws.i = 1;
64     ws.s = "abc";
65     ws.c = 'x';
66     dump_bytes(ws, "WithString");

```

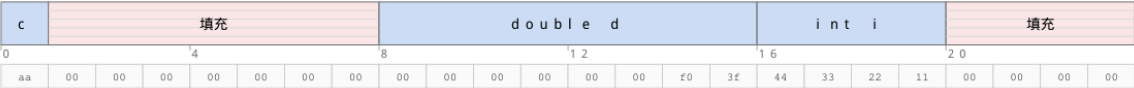
```
67 |     printf(" → 前 4 字节是 i，3 字节填充，再往后 24 字节是 string 本体。\\n");
68 |     return 0;
69 | }
```

p1_02_layout.cpp • 内存结构图

按本机实测绘制 (Apple clang 2.1 / arm64 / macOS) : 1 格 = 1 字节。上排是字段归属, 中间是偏移标尺, 下排是实测 dump 出的原始字节。

数据成员 填充 (内容未定义) 库类型内部 标志位 / 指针字段

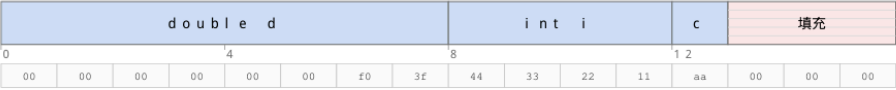
LooseLayout char → double → int (顺序不好) sizeof = 24 字节



下排 = 本例 dump 出的原始字节 (小端; ?? = 填充内容未定义, 实测每次运行都不同)

- double 要求 8 对齐: char 占偏移 0, double 只能从偏移 8 开始, 中间 7 字节变成填充; int 落在偏移 16, 尾部再补 4 → 24。
- 字节全对得上: aa 是 char; f0 3f 是 double 1.0 的高位 (小端); 44 33 22 11 是 int 0x11223344。
- 这里的填充显示为 00, 是因为 LooseLayout 是平凡类型, `l {}` 值初始化会连填充位一起清零; 换成含非平凡成员的 WithString, {} 就不再清零填充位 —— 下一页就是反例。

TightLayout double → int → char (调整后) sizeof = 16 字节



下排 = 本例 dump 出的原始字节 (小端; ?? = 填充内容未定义, 实测每次运行都不同)

- 同样三个成员: 8 + 4 + 1 + 3 = 16; 填充被挤到尾部, 只剩 3 字节。
- 规则: 成员按自身对齐落位, 结构体整体按最大成员对齐 —— 把大的放前面、小的挤到最后, 就能把填充集中到尾部一块。

同样的三个成员, 只改声明顺序

LooseLayout (char 打头) 24 字节

TightLayout (double 打头) 16 字节

- 同样三个成员、同样的值, 只改声明顺序: 24 → 16, 省 8 字节 (约 33%)。
- Rust 默认会重排字段来省内存, C++ 不会: 手动排序是 C++ 程序员的基本功。

三条规则

1. 成员按自身对齐要求落位: char = 1、int = 4、double = 8、指针 = 8; 放不下就往右挪, 空洞变成填充。

2. 结构体整体按最大成员对齐, 尾部常要补一段, 保证数组里每个元素都对齐 (LooseLayout 是 8 对齐、TightLayout 也是 8 对齐)。

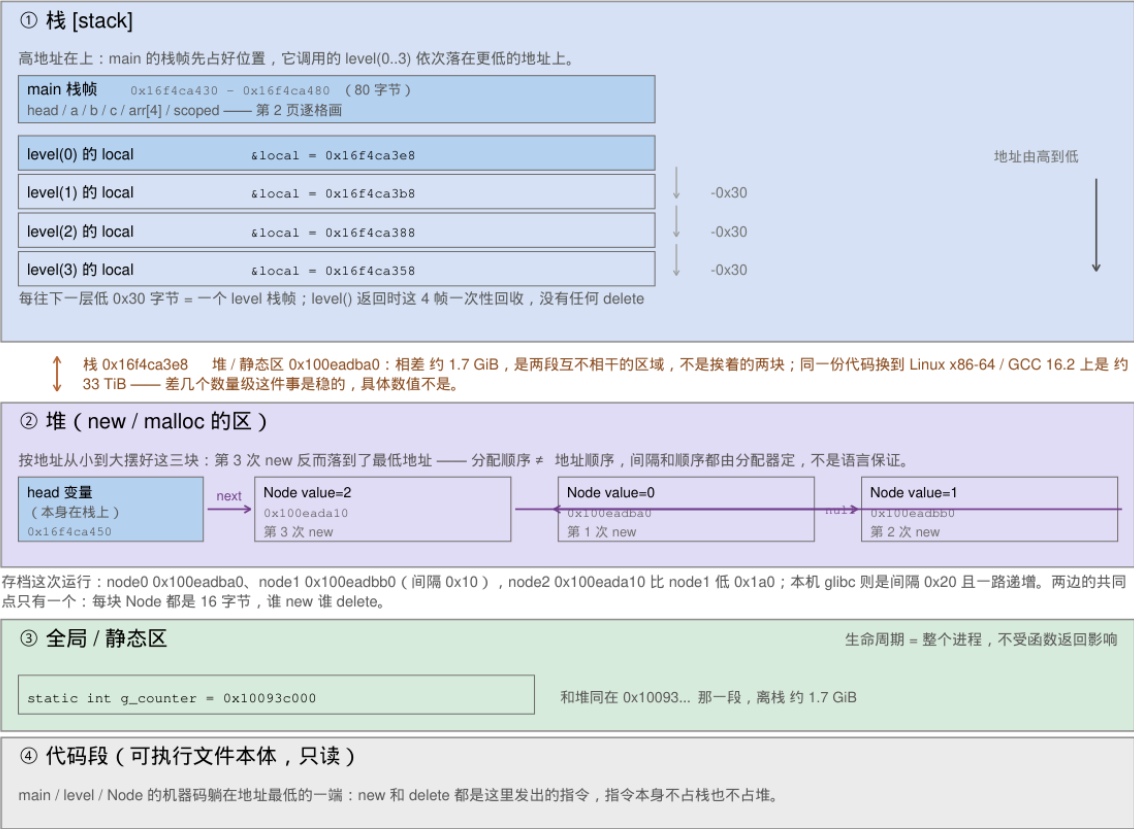
3. 填充属于对象的一部分 (sizeof 算进去), 但它的内容未定义: 不要 memcmp 比较含填充的结构体, 也不要按字节序列化后跨平台传 —— 这正是宿主层与前端交换数据时要先定协议的原因。


```
1 // 例题 1-3: 栈与堆的本质区别 (地址方向、生命周期、谁负责释放)
2 #include <stdio>
3
4 static int g_counter = 0;
5
6 static void level(int depth) {
7     for (; depth <= 3; ++depth) {
8         int local = depth;
9         printf("level(%d): 栈变量 &local = %p\n", depth, (void *)&local);
10    }
11 }
12
13 struct Node {
14     int value;
15     Node *next;
16 };
17
18 int main() {
19     printf("-- 栈: 向低地址方向增长 --\n");
20     level(0);
21     printf("g_counter (全局/静态区) 地址 = %p\n", (void *)&g_counter);
22
23     printf("-- 同一函数里的局部变量 --\n");
24     int a = 1, b = 2, c = 3;
25     printf("&a=%p &b=%p &c=%p (地址不保证连续) \n", (void *)&a, (void *)&b,
26           (void *)&c);
27     int arr[4] = {0, 1, 2, 3};
28     for (int i = 0; i < 4; ++i) {
29         printf("arr[%d] 值=%d 地址=%p (同一数组内连续, 步长 %zu) \n", i, arr[i],
30               (void *)&arr[i], sizeof(int));
31     }
32
33     printf("\n-- 堆: 地址由分配器决定, 通常与栈相距很远 --\n");
34     Node *head = nullptr;
35     for (int i = 0; i < 3; ++i) {
36         Node *node = new Node{i, head};
37         printf("new Node{value=%d} → 堆地址 %p\n", i, (void *)node);
38         head = node;
39     }
40     printf("头指针 head 本身在栈上: &head=%p (%zu 字节), 指向堆上的节点\n",
41           (void *)&head, sizeof(head));
42
43     printf("\n-- 生命周期 --\n");
44     {
45         int scoped = 42;
46         printf("作用域内的 scoped 在 %p\n", (void *)&scoped);
47     }
48     int reused = 7;
49     printf("作用域外的 reused 在 %p (地址常被复用, 说明栈空间是重复利用的) \n",
50           (void *)&reused);
51
52     printf("\n-- 释放: 栈自动, 堆手动 --\n");
53     while (head) {
54         Node *next = head->next;
55         printf("delete 堆节点 %p (值 %d) \n", (void *)head, head->value);
56         delete head;
57         head = next;
58     }
59     return 0;
60 }
```

p1_03_stack_heap.cpp · 内存结构图

数据源：code/outputs/p1_03_stack_heap.txt（Apple clang 21 / arm64 / macOS 的存档实验输出）。地址每次运行都不同 —— 看相对关系，不看具体数值。

栈（自动回收） 堆（要手动 delete） 全局 / 静态区 代码段 本次没用到的位置



p1_03_stack_heap.cpp · 内存结构图 (2/3) · main 的栈帧

同一个函数里的变量全在 main 的栈帧里。下面从 0x16f4ca430 起、4 字节一格画出来，格子内的地址来自存档的同一运行。

普通局部变量 作用域一结束就被复用 指针变量（指向堆） 这次没打印到的位置

main 的栈帧俯视图 4 字节一格，共 20 格 = 80 字节



- arr[0..3] 四格紧挨着、步长恰好 4 = sizeof(int)：数组是一个完整的对象，元素必须连续存放。
- a / b / c 也挨着，但谁在高位完全由编译器决定：这份存档是递减的 (0x16f4ca468 > 0x16f4ca464 > 0x16f4ca460)，本机 GCC 却是递增的 (a < b < c) —— 别按声明顺序推地址。
- head 占两格：它是个 8 字节指针，自己老老实实待在栈上，指向的对象在堆上 —— 「变量」和「变量指向的东西」不一定在同一个区。
- 灰色格子是这次运行没打印到的位置：对齐和调用约定要临时值留空档，所以栈帧里总有不属于任何变量的字节。
- reused 落在 scoped 用过的位置上（存档里两者相邻 4 字节，本机 GCC 实测地址完全相同）：作用域一结束，那块栈空间马上被别人接手。

递归：每层一个固定大小的栈帧 同一个 level() 反复调用，地址一路往下走

调用层	&local	与上一层差	说明
level (0)	0x16f4ca3e8	-	最先调用，地址最高
level (1)	0x16f4ca3b8	-0x30	低一个栈帧
level (2)	0x16f4ca388	-0x30	低一个栈帧
level (3)	0x16f4ca358	-0x30	低一个栈帧

- 每调用一层，栈指针往下走 0x30 字节：一个 int + 保存的返回地址 / 寄存器 + 对齐。
- 动手改一改：给 level() 加一个 char buf[64]，本机实测每层差从 0x30 变成 0x70 = 112 字节 —— 栈帧大小 = 局部变量之和（对齐之后）。
- 四层一共才 0xc0 字节；把深度换成十万量级就是栈溢出（第三阶段例题 3-3 的伏笔）。

生命周期：作用域一结束，地址就被接管



- 栈的「释放」不是清空内存，只是把栈指针挪回去：值还躺在那儿，指针还在你手上，那就是野指针。
- 同一份代码换到本机 GCC，scoped 和 reused 实测落在完全相同的地址 —— 复用不是巧合，是规则。

动手改一改 先预测，再跑，最后回到这张图上对一对

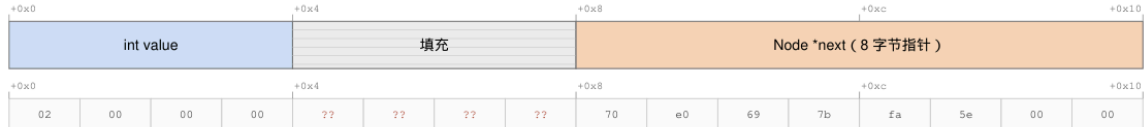
- 给 level() 加一个 char buf[64]：先算出每层差会变成多少（提示：向上取整到 16 的倍数），再跑 —— 本机实测是 0x70 = 112 字节。
- 把三次 new Node 改成 new Node[3]：堆上从 3 次分配变成 1 次、地址一定连续，释放也从 3 次 delete 变成 1 次 delete[] —— 这是唯一能「保证连续」的写法。
- 把 a / b / c 的声明顺序改一改，看地址跟不跟着换：验证「顺序由编译器定」，而不是「按声明顺序排」。

p1_03_stack_heap.cpp · 内存结构图 (3/3) · 堆上的 Node 与释放

堆这一侧的三个问题：一个 Node 占多少字节、三者怎么连起来、谁负责把它还回去。

■ 数据成员 ■ 对齐填充（内容未定义） ■ 指针字段 ■ 堆上的对象

Node 的 16 字节 8 字节的数据 + 8 字节的代价（存档与本机实测一致）



下排是本机 dump 出来的 16 个原始字节（小端）：02 00 00 00 是 value，紧跟着的 8 个字节是指向下一节点的地址；中间 4 个 ?? 是填充，内容未定义，这次刚好是 00。

- int value 在偏移 0；next 是 8 字节指针、要求 8 对齐，所以 4..7 只能当填充——和例题 1-2 的填充是同一件事，只不过这次它让每个 Node 从 12 字节变成 16 字节。
- 结构体布局由 ABI 定，跨平台反而很稳：本机 GCC 与存档的 clang 都是 16 字节、value 在 0、next 在 8。

分配 → 链表 → 释放：每一步都不按地址来 存档这一次运行的完整顺序

步骤	发生了什么	地址
new #1	node0 : value=0, next=nullptr (head 当时还是空)	0x100eadba0
new #2	node1 : value=1, next=node0	0x100eadbb0
new #3	node2 : value=2, next=node1, 地址反而最低	0x100eada10
delete #1	先存下 head->next, 再 delete	0x100eada10
delete #2	head 前移到 node1, 重复上一步	0x100eadbb0
delete #3	head 变成 nullptr, 循环结束	0x100eadba0

```
while (head) { Node *next = head->next; delete head; head = next; }
```

- 必须先存 next：delete head 之后再读 head->next 就是读已释放内存（ASan 会报 heap-use-after-free，第三阶段例题 3-4）。
- 释放顺序 2 → 1 → 0，正好是 new 的倒序；栈上那些变量不用管，main 一返回全消失，堆上这三个必须有人 delete——漏一个就是泄漏。

换一台机器复核：结论稳、数值不稳 左列是仓库存档（macOS / arm64），右列是本机跑 probe_p1_03.cpp 的结果

观察项	macOS / Apple clang 21（存档）	Linux x86-64 / GCC 16.2（本机）
递归每层差	0x30	0x30
加 char buf[64] 后每层差	未测	0x70
Node 大小 / 偏移	16 字节，value@0、next@8	16 字节，value@0、next@8
三块堆的间隔与顺序	0x10，第 3 块回到更低地址	0x20，单调递增
a / b / c 地址顺序	递减（a 最高）	递增（a 最低）
scoped 与 reused	相邻 4 字节	地址完全相同
栈 静态区距离	约 1.7 GiB	约 33 TiB

- 两列里唯一稳的是「结构」：栈向下、每层一个固定栈帧、Node 16 字节、指针在栈而对象在堆。
- 数值每次运行都在变（地址、间隔、距离都会被 ASLR 和分配器改掉），所以只能记规律、不能记数字——这是这道例题最实用的一条。

栈 vs 堆：一张对照表 本例题标题里的「本质区别」就是这六行

对比项	栈	堆
空间从哪来	进入函数时由编译器预留	new / malloc 向分配器申请
谁释放、什么时候	运行时自动：作用域结束 / 函数返回	你：delete / free，可提前也可延后
释放晚了	——	泄漏：长跑程序内存只涨不落
释放早了 / 越界	踩到相邻栈变量，现象随运行改变	野指针、double free（例题 3-4、3-5）
地址规律	必然向下，每层一个固定大小的帧	由分配器决定：可高可低、间隔不定
容量上限	通常 8 MB 量级（本机 ulimit -s = 8192）	受物理内存 + 交换空间限制

三条结论

1. 栈和堆的差别不是「快慢」，是「谁负责」：栈由编译器和运行时按作用域自动回收，堆必须由你 delete——漏了是泄漏，早删了是野指针。
2. 地址的方向与顺序都不是语言保证：栈一定向下长、每层一个固定栈帧；堆的地址和间隔看分配器，甚至可能像存档里那样「后分配的反而更低」。
3. 知道这些是为了做对两件事：跨语言传结构体时按 ABI 而不是靠猜来算大小；写宿主层接口时先想清楚「谁拥有、活到什么时候」，再动手写代码。

```

1 // 例题 1-4: 虚函数如何改变对象内存 (vptr 与虚表)
2 // 观察点: vptr 在对象最前面; 虚表是每个类一张全局表, 对象只存一个指针
3 #include <cstdio>
4
5 struct Plain {
6     int value;
7     void f() { printf("Plain::f()\n"); }
8 };
9
10 struct WithVirtual {
11     int value;
12     virtual void f() { printf("WithVirtual::f()\n"); }
13     virtual void g(int x) { printf("WithVirtual::g(%d)\n", x); }
14 };
15
16 struct Derived : WithVirtual {
17     void f() override { printf("Derived::f()\n"); }
18 };
19
20 static void dump_vtable(void *obj, const char *name, int slots) {
21     void **vptr = *reinterpret_cast<void ***>(obj);
22     printf("%-15s vptr=%p\n", name, (void *)vptr);
23     for (int i = 0; i < slots; ++i) printf("    vtable[%d] = %p\n", i, vptr[i]);
24 }
25
26 int main() {
27     printf("sizeof(Plain)      = %zu (int + 填充) \n", sizeof(Plain));
28     printf("sizeof(WithVirtual) = %zu (int 4 + 填充 4 + vptr 8) \n", sizeof(WithVirtual));
29     printf("sizeof(Derived)      = %zu (没加成员, 只是换掉表里的函数) \n\n", sizeof(Derived));
30
31     WithVirtual base;
32     WithVirtual base2;
33     Derived derived;
34     dump_vtable(&base, "WithVirtual", 2);
35     dump_vtable(&base2, "WithVirtual#2", 2);
36     dump_vtable(&derived, "Derived", 2);
37     printf("→ 同类对象共用同一张表; 子类重写后表里的函数地址不同。 \n\n");
38
39     printf("对象前 8 字节的原始值 (就是 vptr) : ");
40     unsigned char *raw = reinterpret_cast<unsigned char *>(&base);
41     for (int i = 0; i < 8; ++i) printf("%02x", raw[i]);
42     printf("\n\n-- 绕过语法, 直接按虚表调用 (多态的机械本质) --\n");
43     void **vptr = *reinterpret_cast<void ***>(&base);
44     auto f0 = reinterpret_cast<void (*)>(WithVirtual *)>(vptr[0]);
45     auto f1 = reinterpret_cast<void (*)>(WithVirtual *, int)>(vptr[1]);
46     f0(&base);
47     f1(&base, 42);
48     return 0;
49 }

```

```
1 // 例题 1-5: 多态 = 运行时通过 vptr 查表; 非虚函数 = 编译期按指针类型绑定
2 #include <stdio>
3
4 struct Base {
5     virtual void speak() { printf("Base::speak()\n"); }
6     void name() { printf("Base::name() ← 非虚, 看指针的静态类型\n"); }
7     virtual ~Base() { printf("~Base()\n"); }
8 };
9
10 struct Derived : Base {
11     void speak() override { printf("Derived::speak()\n"); }
12     void name() { printf("Derived::name() ← 非虚, 看指针的静态类型\n"); }
13     virtual void extra() { printf("Derived::extra() ← 新增虚函数, 追加在表尾\n"); }
14     ~Derived() override { printf("~Derived()\n"); }
15 };
16
17 int main() {
18     Derived d;
19     Base *pb = &d;
20     Derived *pd = &d;
21     printf("&d=%p pb=%p pd=%p (单继承下三者地址相同) \n", (void *)&d, (void *)pb, (void *)pd);
22
23     printf("\n-- 虚函数: 运行时按对象真实类型查表 --\n");
24     pb->speak();
25     pd->speak();
26
27     printf("\n-- 非虚函数: 编译期按指针类型绑定 --\n");
28     pb->name();
29     pd->name();
30
31     printf("\n-- 打印两张不同的虚表 --\n");
32     Base fake;
33     void **base_vp
```

```
1 // 例题 2-1: std::string 的 SSO、容量与扩容
2 // 观察点: 小字符串的缓冲就在对象内部; 越界那一刻数据会搬到堆上
3 #include <stdint>
4 #include <stdio>
5 #include <string>
6
7 static void show(const std::string &s, const char *label) {
8     const char *data = s.data();
9     const char *self = reinterpret_cast<const char *>(&s);
10    bool inside = data >= self && data < self + sizeof(s);
11    printf("%-10s size=%2zu capacity=%2zu data=%p SSO=%s\n",
12          label, s.size(), s.capacity(), (const void *)data, inside ? "内部缓冲" : "堆上");
13 }
14
15 int main() {
16     printf("sizeof(std::string) = %zu (libc++ : 指针/长度/容量 24 字节, SSO 缓冲嵌在里面) \n\n",
17           sizeof(std::string));
18
19     std::string shortStr = "hi";
20     std::string mediumStr = "01234567890123456789"; // 20 字符
21     std::string longStr = "0123456789012345678901234567890"; // 31 字符
22     show(shortStr, "短字符串");
23     show(mediumStr, "20 字符");
24     show(longStr, "31 字符");
25     printf("→ libc++ 的 SSO 上限是 22 个字符: 不超过就放在对象内部, 不碰堆。 \n\n");
26
27     printf("-- 逐步增长, 看容量怎么跳 --\n");
28     std::string s;
29     for (int i = 0; i < 40; ++i) {
30         s += 'x';
31         if (i < 4 || i == 20 || i == 21 || i == 22 || i == 23 || i == 39) {
32             printf("长度 %2d : capacity=%2zu data=%p\n", (int)s.size(), s.capacity(),
33                   (const void *)s.data(),
34                   s.capacity() > 22 ? " ← 已经在堆上了" : "");
35         }
36     }
37     printf("\n-- 扩容会重新分配 + 拷贝, 预留容量可避免 --\n");
38     std::string r;
39     r.reserve(64);
40     const char *before = r.data();
41     for (int i = 0; i < 40; ++i) r += 'y';
42     printf("reserve(64) 后 : capacity=%zu 追加 40 次, data 地址%s\n",
43           r.capacity(), r.data() == before ? "没变 (一次分配搞定)" : "变了 (发生了重分配)");
44     return 0;
45 }
```

```

1 // 例题 2-2: vector 的连续内存、扩容倍数、迭代器/指针失效
2 #include <stdio>
3 #include <vector>
4
5 struct Tracked {
6     int value;
7     static int alive;
8     static int copies;
9     static int moves;
10    explicit Tracked(int v) : value(v) { ++alive; }
11    Tracked(const Tracked &other) : value(other.value) { ++alive; ++copies; }
12    Tracked(Tracked &&other) noexcept : value(other.value) { ++alive; ++moves; }
13    ~Tracked() { --alive; }
14 };
15 int Tracked::alive = 0;
16 int Tracked::copies = 0;
17 int Tracked::moves = 0;
18
19 int main() {
20     printf("-- 基本类型: 扩容时 size/capacity/地址怎么变 --\n");
21     std::vector<int> v;
22     const void *last = nullptr;
23     for (int i = 0; i < 17; ++i) {
24         v.push_back(i);
25         if (v.data() != last) {
26             printf("push_back(%2d) → size=%2zu capacity=%2zu data=%p ← 发生重分配\n",
27                 i, v.size(), v.capacity(), (const void *)v.data());
28             last = v.data();
29         }
30     }
31     printf("容量序列大致是 1,2,4,8,16,32: libc++ 按 2 倍增长, 均摊 O(1)。 \n\n");
32
33     printf("-- 元素是自定义类型: 搬家的代价能被看见 --\n");
34     {
35         std::vector<Tracked> items;
36         for (int i = 0; i < 8; ++i) {
37             items.emplace_back(i);
38         }
39         printf("插入 8 个元素后: 存活=%d 拷贝=%d 移动=%d\n",
40             Tracked::alive, Tracked::copies, Tracked::moves);
41         printf("→ 扩容时用移动 (Tracked 有 noexcept 移动构造), 拷贝次数为 0 才是合格实现。 \n");
42     }
43     printf("vector 离开作用域后: 存活=%d (全部析构, 没有泄漏) \n\n", Tracked::alive);
44
45     printf("-- 迭代器/指针失效 --\n");
46     std::vector<int> w{1, 2, 3};
47     const int *p = w.data();
48     printf("扩容前 data=%p\n", (const void *)p);
49     w.push_back(4);
50     printf("push_back 后 data=%p, 旧指针 %s\n", (const void *)w.data(),
51         p == w.data() ? "仍然有效" : "已悬空 (继续解引用就是 UB, ASan 会抓到)");
52     w.reserve(64);
53     printf("reserve 之后 data=%p, 之前那个指针同样失效\n", (const void *)w.data());
54     printf("\n结论: 只要发生重分配, 所有指向元素的指针/引用/迭代器全部作废。 \n");
55     return 0;
56 }

```

```

1 // 例题 2-3: RAII —— 把"释放"这件事交给析构函数
2 // 观察点: 无论正常返回、提前 return 还是抛异常, 析构都会执行, 账本一定归零
3 #include <cstddef>
4 #include <cstdio>
5 #include <stdexcept>
6 #include <utility>
7 #include <vector>
8
9 static int g_outstanding = 0; // 当前还没释放的堆分配数 (内存账本)
10
11 class Buffer {
12 public:
13     Buffer(std::size_t n, const char *tag)
14         : tag_(tag), size_(n), data_(new unsigned char[n]) {
15         ++g_outstanding;
16         printf(" 构造 %-8s: 申请 %zu 字节 @ %p (账本=%d) \n", tag_, n, (void *)data_, g_outstanding);
17     }
18     Buffer(const Buffer &) = delete; // 独占资源, 禁止拷贝
19     Buffer &operator=(const Buffer &) = delete;
20     Buffer(Buffer &&other) noexcept // 只允许移动: 转移所有权
21         : tag_(other.tag_), size_(other.size_), data_(other.data_) {
22         other.data_ = nullptr;
23         other.size_ = 0;
24         printf(" 移动 %-8s: 指针移交给新对象 (账本不变=%d) \n", tag_, g_outstanding);
25     }
26     ~Buffer() {
27         if (data_) {
28             delete[] data_;
29             --g_outstanding;
30             printf(" 析构 %-8s: 释放 @ %p (账本=%d) \n", tag_, (void *)data_, g_outstanding);
31         }
32     }
33     const char *tag() const { return tag_; }
34     std::size_t size() const { return size_; }
35
36 private:
37     const char *tag_;
38     std::size_t size_;
39     unsigned char *data_;
40 };
41
42 static void raiiVersion() {
43     printf("[RAII 版本]\n");
44     Buffer first(1024, "first");
45     Buffer second(2048, "second");
46     printf(" → 出作用域时按构造的逆序析构: \n");
47 }
48
49 static void rawVersion() {
50     printf("[裸指针版本]\n");
51     unsigned char *raw = new unsigned char[1024];
52     ++g_outstanding;
53     printf(" new 了 1024 字节 @ %p, 但忘了 delete\n", (void *)raw);
54 }
55
56 static void throwVersion() {
57     printf("[中途抛异常]\n");
58     Buffer guard(512, "guard");
59     throw std::runtime_error("模拟失败");
60 }
61
62 int main() {
63     raiiVersion();
64     printf(" 账本 = %d\n\n", g_outstanding);
65
66     rawVersion();

```

```
67     printf(" 账本 = %d ← 这 1 就是泄漏，ASan/LSan 会报 leak\n\n", g_outstanding);
68     g_outstanding = 0; // 演示用：人为清账，避免影响后面的输出
69
70     try {
71         throwVersion();
72     } catch (const std::exception &e) {
73         printf(" 捕获异常：%s\n\n", e.what());
74     }
75     printf(" 账本 = %d (异常路径也被清理干净) \n\n", g_outstanding);
76
77     printf("[把 Buffer 放进 vector：只移动，不拷贝]\n");
78     {
79         std::vector<Buffer> pool;
80         pool.reserve(2);
81         pool.emplace_back(128, "pool-1");
82         pool.emplace_back(256, "pool-2");
83         pool.emplace_back(512, "pool-3"); // 触发扩容：已有元素被移动
84         printf(" vector 内元素数=%zu，账本=%d\n", pool.size(), g_outstanding);
85     }
86     printf(" vector 销毁后账本 = %d\n", g_outstanding);
87     return 0;
88 }
```

```

1 // 例题 2-4: unique_ptr —— 编译器就禁止“共享”，对齐 Rust 的所有权直觉
2 #include <cstdio>
3 #include <memory>
4 #include <utility>
5 #include <vector>
6
7 struct Resource {
8     int id;
9     explicit Resource(int i) : id(i) { printf(" 构造 Resource(%d)\n", id); }
10    ~Resource() { printf(" 析构 Resource(%d)\n", id); }
11 };
12
13 static void consume(std::unique_ptr<Resource> p) { // 按值接收 = 拿走所有权
14     printf(" consume 里看到的 id=%d\n", p->id);
15 } // 离开函数即析构
16
17 static void observe(const std::unique_ptr<Resource> &p) { // 借用，不转移
18     printf(" observe 里看到的 id=%d (所有权还在调用方)\n", p->id);
19 }
20
21 int main() {
22     printf("-- 构造与自动释放 --\n");
23     {
24         auto p = std::make_unique<Resource>(1);
25         observe(p);
26         printf(" 离开作用域 → 自动 delete\n");
27     }
28
29     printf("\n-- 移动：所有权转移，源指针变空 (Rust 的 move 同款语义) --\n");
30     {
31         auto owner = std::make_unique<Resource>(2);
32         printf(" move 前 owner=%p\n", (void *)owner.get());
33         auto thief = std::move(owner);
34         printf(" move 后 owner=%p (空) thief=%p\n", (void *)owner.get(), (void *)thief.get());
35         if (!owner) printf(" 对空 unique_ptr 解引用是 UB，只能先判空。\\n");
36     }
37
38     printf("\n-- 传参把所有权交出去 --\n");
39     consume(std::make_unique<Resource>(3));
40
41     printf("\n-- 放进容器：只移动，指针稳定 --\n");
42     {
43         std::vector<std::unique_ptr<Resource>> pool;
44         pool.reserve(3);
45         for (int i = 4; i <= 6; ++i) pool.push_back(std::make_unique<Resource>(i));
46         printf(" 容器里 %zu 个对象，析构顺序与构造相反：\\n", pool.size());
47     }
48
49     printf("\n-- 数组形式与自定义删除器 --\n");
50     {
51         auto arr = std::make_unique<int[]>(4);
52         arr[0] = 42;
53         printf(" unique_ptr<int[]> arr[0]=%d (析构用 delete[]) \\n", arr[0]);
54         auto closer = std::unique_ptr<FILE, int (*)>(fopen("/dev/null", "r"), &fclose);
55         printf(" 自定义删除器：FILE* = %s\\n", closer ? "已打开" : "打开失败");
56     }
57     printf("\n全部作用域结束，所有资源都已释放。\\n");
58     return 0;
59 }

```

```

1 // 例题 2-5: shared_ptr 的引用计数、循环引用, 以及 weak_ptr 怎么破坏
2 #include <cstdio>
3 #include <memory>
4
5 struct Node {
6     int id;
7     std::shared_ptr<Node> next; // 强引用: 会推高计数
8     std::weak_ptr<Node> back; // 弱引用: 不增加计数
9     explicit Node(int i) : id(i) { printf(" 构造 Node(%d)\n", id); }
10    ~Node() { printf(" 析构 Node(%d)\n", id); }
11 };
12
13 static void countDemo() {
14     printf("[引用计数变化]\n");
15     auto a = std::make_shared<Node>(1);
16     printf(" make_shared 后 use_count=%ld\n", a.use_count());
17     {
18         auto b = a;
19         printf(" 拷贝一份后 use_count=%ld\n", a.use_count());
20     }
21     printf(" 副本销毁后 use_count=%ld\n", a.use_count());
22     auto c = std::move(a);
23     printf(" move 之后: 源=%s, 新=%ld\n", a ? "非空" : "空", c.use_count());
24 }
25
26 static void cycleLeak() {
27     printf("\n[循环引用: 两个 shared_ptr 互指 → 谁都释放不了]\n");
28     auto x = std::make_shared<Node>(2);
29     auto y = std::make_shared<Node>(3);
30     x->next = y;
31     y->next = x;
32     printf(" 离开作用域前: use_count x=%ld y=%ld\n", x.use_count(), y.use_count());
33     printf(" → 你会看到下面没有析构输出, 说明对象泄漏了 (ASan 能抓到)\n");
34 }
35
36 static void fixedCycle() {
37     printf("\n[用 weak_ptr 破坏: back 只是观察者]\n");
38     auto x = std::make_shared<Node>(4);
39     auto y = std::make_shared<Node>(5);
40     x->next = y;
41     y->back = x; // 弱引用不推高计数
42     printf(" 离开作用域前: x.use_count=%ld y.use_count=%ld\n", x.use_count(), y.use_count());
43     if (auto locked = y->back.lock()) printf(" 弱引用能临时提升为强引用, 看到 Node(%d)\n", locked->id);
44 }
45
46 int main() {
47     countDemo();
48     cycleLeak();
49     printf(" (上面没有 Node(2)/Node(3) 的析构 —— 计数永远不归零)\n");
50     fixedCycle();
51     printf("\nNode(4)/Node(5) 会在离开作用域时正常析构。");
52     return 0;
53 }

```

```

1 // 例题 3-1: 多继承的内存布局 —— 一个对象里为什么会有两个 vptr
2 #include <stdio>
3
4 struct A {
5     int a = 1;
6     virtual void fa() { printf("A::fa()\n"); }
7     virtual ~A() = default;
8 };
9
10 struct B {
11     int b = 2;
12     virtual void fb() { printf("B::fb()\n"); }
13     virtual ~B() = default;
14 };
15
16 struct C : A, B {
17     int c = 3;
18     void fa() override { printf("C::fa() (重写 A 的)\n"); }
19     void fb() override { printf("C::fb() (重写 B 的)\n"); }
20     ~C() override = default;
21 };
22
23 int main() {
24     printf("sizeof(A)=%zu sizeof(B)=%zu sizeof(C)=%zu\n", sizeof(A), sizeof(B), sizeof(C));
25     printf("→ C = A 子对象(vp+int, 16) + B 子对象(vp+int, 16) + int c + 填充\n\n");
26
27     C obj;
28     A *pa = &obj;
29     B *pb = &obj;
30     C *pc = &obj;
31     printf("同一个对象，三个指针指向不同的位置:\n");
32     printf(" pc = %p (对象首地址) \n", (void *)pc);
33     printf(" pa = %p (A 子对象, 偏移 %ld) \n", (void *)pa, (long)((char *)pa - (char *)pc));
34     printf(" pb = %p (B 子对象, 偏移 %ld) \n", (void *)pb, (long)((char *)pb - (char *)pc));
35
36     printf("\n两个子对象各有一张虚表:\n");
37     printf(" A 子对象的 vptr = %p\n", *reinterpret_cast<void **>(pa));
38     printf(" B 子对象的 vptr = %p\n", *reinterpret_cast<void **>(pb));
39
40     printf("\n通过不同指针调用，都是虚调用，各自查各自的表:\n");
41     pa->fa();
42     pb->fb();
43
44     printf("\n-- 指针调整是编译器做的，不是免费的 --\n");
45     B *fromC = static_cast<B *>(pc); // 正确: C 明确继承 B, 编译器自动偏移
46     B *wrong = reinterpret_cast<B *>(pa); // 危险: 按字节重解释, 地址没有偏移
47     B *cross = dynamic_cast<B *>(pa); // 运行时跨类转换 (A 多态才允许)
48     printf(" static_cast<B*>(pc) = %p (正确, 从对象首地址偏移到 B 子对象) \n", (void *)fromC);
49     printf(" reinterpret_cast<B*>(pa) = %p (错, 等于把 A 子对象当 B 用 → UB) \n", (void *)wrong);
50     printf(" dynamic_cast<B*>(pa) = %p (正确, 代价是运行时检查) \n", (void *)cross);
51
52     printf("\n-- 用基类指针 delete: 虚析构保证完整析构 --\n");
53     A *heap = new C();
54     printf(" delete 前先看 pb 偏移: B* p = %p\n", (void *)dynamic_cast<B *>(heap));
55     delete heap;
56     return 0;
57 }

```

```

1 // 例题 3-2: 菱形继承与虚继承 —— 两份 Base 还是一份 Base
2 #include <stdio>
3
4 struct Base {
5     int value = 10;
6     virtual void show() { printf("Base::show() value=%d\n", value); }
7     virtual ~Base() = default;
8 };
9
10 // 不用虚继承: D 里有两份 Base
11 struct Left : Base {};
12 struct Right : Base {};
13 struct Diamond : Left, Right {};
14
15 // 用虚继承: 只剩一份 Base, 编译器插入 vbptr/偏移表来定位它
16 struct VLeft : virtual Base {};
17 struct VRight : virtual Base {};
18 struct VShape : VLeft, VRight {
19     void show() override { printf("VShape::show() value=%d (唯一一份 Base) \n", value); }
20 };
21
22 int main() {
23     printf("sizeof(Base)=%zu sizeof(Left)=%zu sizeof(Diamond)=%zu\n",
24           sizeof(Base), sizeof(Left), sizeof(Diamond));
25     printf("→ Diamond 里有两份 Base (2 个 vbptr + 2 个 int) \n");
26     printf("sizeof(VLeft)=%zu sizeof(VRight)=%zu sizeof(VShape)=%zu\n",
27           sizeof(VLeft), sizeof(VRight), sizeof(VShape));
28     printf("→ 虚继承后只有一份 Base, 多出来的是指向公共 Base 的 vbptr\n\n");
29
30     Diamond d;
31     printf("[非虚继承]\n");
32     d.Left::value = 1;
33     d.Right::value = 2;
34     printf(" d.Left::value=%d 落在 %p\n", d.Left::value, (void *)&d.Left::value);
35     printf(" d.Right::value=%d 落在 %p\n", d.Right::value, (void *)&d.Right::value);
36     printf(" 两个 Base 子对象地址相差 %ld 字节, 写 d.value 会报二义性编译错误。 \n\n",
37           (long)((char *)&d.Right::value - (char *)&d.Left::value));
38
39     VShape s;
40     printf("[虚继承]\n");
41     s.value = 99; // 只有一份, 不需要限定名
42     printf(" &s.value = %p, VLeft* 与 VRight* 指向的公共 Base 相同: %s\n",
43           (void *)&s.value,
44           (void *)static_cast<Base *>(static_cast<VLeft *>(&s)) ==
45           (void *)static_cast<Base *>(static_cast<VRight *>(&s))
46           ? "是"
47           : "否");
48     printf(" 通过左路指针调用虚函数: \n");
49     static_cast<VLeft *>(&s)->show();
50
51     printf("\n[内存布局速查]\n");
52     printf(" 非虚菱形: Diamond{ Left{ vbptr, Base{...}? }, Right{...} } → 2 份数据, 改一处另一处不动\n");
53     printf(" 虚菱形: VShape{ vbptr..., 公共 Base 一份 } → 只有一份数据, 代价是每次访问多一次间接寻址\n");
54     return 0;
55 }

```

```
1 // 例题 3-3: 堆越界写 —— 编译器不拦, ASan 一抓一个准
2 // 编译: clang++ -std=c++17 -g -00 -fsanitize=address p3_03_overflow.cpp -o p3_03
3 #include <cstdio>
4
5 int main() {
6     int *arr = new int[4];
7     for (int i = 0; i < 4; ++i) arr[i] = i;
8     printf("正常写完 4 个元素: arr[3]=%d\n", arr[3]);
9     printf("下面故意越界写 arr[4]……\n");
10    fflush(stdout);
11    arr[4] = 99; // ← 越界: ASan 会在这里报 heap-buffer-overflow
12    printf("这行几乎不会执行到: %d\n", arr[4]);
13    delete[] arr;
14    return 0;
15 }
```

```
1 // 例题 3-4: 释放后使用 (野指针) —— 内存已经还给分配器, 读到的可能是任意值
2 // 编译: clang++ -std=c++17 -g -O0 -fsanitize=address p3_04_use_after_free.cpp -o p3_04
3 #include <cstdio>
4
5 int main() {
6     int *p = new int(7);
7     printf("分配: p=%p 值=%d\n", (void *)p, *p);
8     delete p;
9     printf("已 delete p, 但指针变量还在 (这就是野指针) \n");
10    fflush(stdout);
11    printf("读野指针: *p=%d\n", *p); // ← ASan 报 heap-use-after-free
12    p = nullptr; // 好习惯: 释放后置空
13    if (!p) printf("置空后可以安全判断: p == nullptr\n");
14    return 0;
15 }
```

```
1 // 例题 3-5: 重复释放 —— 同一块内存还两次, 堆元数据被破坏
2 // 编译: clang++ -std=c++17 -g -O0 -fsanitize=address p3_05_double_free.cpp -o p3_05
3 #include <cstdio>
4 #include <memory>
5
6 int main() {
7     printf("-- 正面: 所有权写进类型里, 就没有第二次释放的机会 --\n");
8     {
9         auto safe = std::make_unique<int>(2);
10        printf(" unique_ptr 管着 %d, 作用域结束自动释放一次\n", *safe);
11        // delete safe.get(); // 想重复释放得先绕过类型系统: unique_ptr 会再删一次
12    }
13    printf(" 离开作用域, 资源恰好释放一次。\\n\\n");
14
15    printf("-- 反面: 裸指针 delete 两次 --\n");
16    int *raw = new int(1);
17    delete raw;
18    fflush(stdout);
19    delete raw; // ← ASan 报 attempting double-free
20    return 0;
21 }
```

```

1 // 例题 3-6: 综合项目骨架 —— 多态成绩管理 (vector + 智能指针 + 文件 + RAII)
2 // 每个设计点都对应前面某道例题: 多态=3-1、所有权=2-4、异常安全=2-3
3 #include <cstdio>
4 #include <fstream>
5 #include <memory>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9
10 class Student {
11 public:
12     Student(std::string name, double base) : name_(std::move(name)), base_(base) {
13         printf(" 构造 %-8s 对象 @ %p\n", name_.c_str(), (void *)this);
14     }
15     virtual ~Student() { printf(" 析构 %-8s @ %p\n", name_.c_str(), (void *)this); }
16
17     virtual double finalScore() const = 0; // 纯虚: 算法由子类决定
18     virtual const char *kind() const = 0;
19     virtual std::string describe() const { // 基类实现里调用虚函数 → 多态
20         char buffer[128];
21         snprintf(buffer, sizeof(buffer), "%-8s %-8s 卷面 %5.1f → 总评 %5.1f",
22             name_.c_str(), kind(), base_, finalScore());
23         return buffer;
24     }
25     const std::string &name() const { return name_; }
26     double baseScore() const { return base_; }
27
28 protected:
29     std::string name_;
30     double base_;
31 };
32
33 class Regular : public Student {
34 public:
35     using Student::Student;
36     double finalScore() const override { return base_; }
37     const char *kind() const override { return "普通生"; }
38 };
39
40 class Honors : public Student {
41 public:
42     using Student::Student;
43     double finalScore() const override { return base_ * 1.15; } // 加分政策
44     const char *kind() const override { return "优生"; }
45 };
46
47 class Makeup : public Student {
48 public:
49     using Student::Student;
50     double finalScore() const override { return base_ * 0.9; } // 补考折算
51     const char *kind() const override { return "补考生"; }
52 };
53
54 class GradeBook {
55 public:
56     void add(std::unique_ptr<Student> student) { students_.push_back(std::move(student)); }
57
58     void report() const {
59         printf("-- 名册 (按对象真实类型多态求值) --\n");
60         for (const auto &student : students_) {
61             printf(" %s\n", student->describe().c_str());
62         }
63         if (!students_.empty()) {
64             printf(" 平均总评 = %.2f\n", average());
65             void **vptr = *reinterpret_cast<void ***>(students_.front().get());
66             printf(" 首个对象的 vptr = %p (基类指针 + 虚表 → 多态的机械本质) \n", (void *)vptr);

```

```

67     }
68 }
69
70 double average() const {
71     if (students_.empty()) return 0.0;
72     double sum = 0.0;
73     for (const auto &student : students_) sum += student->finalScore();
74     return sum / static_cast<double>(students_.size());
75 }
76
77 bool save(const std::string &path) const {
78     std::ofstream out(path);           // RAII: 离开函数自动关闭
79     if (!out) return false;
80     for (const auto &student : students_) {
81         out << student->kind() << ' ' << student->name() << ' ' << student->baseScore() << '\n';
82     }
83     return static_cast<bool>(out);
84 }
85
86 bool load(const std::string &path) {
87     std::ifstream in(path);
88     if (!in) return false;
89     students_.clear();
90     std::string kind, name;
91     double base = 0.0;
92     while (in >> kind >> name >> base) {
93         if (kind == "优等生") add(std::make_unique<Honors>(name, base));
94         else if (kind == "补考生") add(std::make_unique<Makeup>(name, base));
95         else add(std::make_unique<Regular>(name, base));
96     }
97     return true;
98 }
99
100 private:
101     std::vector<std::unique_ptr<Student>> students_; // 独占所有权，容器只移动
102 };
103
104 int main() {
105     printf("sizeof(Regular)=%zu sizeof(Honors)=%zu sizeof(Makeup)=%zu (都含 vptr) \n\n",
106         sizeof(Regular), sizeof(Honors), sizeof(Makeup));
107
108     const std::string path = "/tmp/cpp_gradebook.txt";
109     {
110         GradeBook book;
111         printf("-- 构建名册 --\n");
112         book.add(std::make_unique<Honors>("Tom", 85));
113         book.add(std::make_unique<Regular>("Jerry", 78));
114         book.add(std::make_unique<Makeup>("Spike", 60));
115         printf("\n");
116         book.report();
117         printf("\n-- 持久化 (ofstream RAII, 无需手动 close) --\n");
118         printf("  保存到 %s : %s\n", path.c_str(), book.save(path) ? "成功" : "失败");
119         printf("\n-- 读回来 (工厂 + unique_ptr 重新建对象) --\n");
120         GradeBook reloaded;
121         if (reloaded.load(path)) {
122             printf("\n");
123             reloaded.report();
124         }
125     } // ← 两个 GradeBook 在这里析构: vector 析构 → unique_ptr 析构 → 虚析构 → 完整释放
126     printf("\n所有对象已按构造的逆序释放，没有裸 new/delete，也没有泄漏。\\n");
127     return 0;
128 }

```

```

1 // 例题 3-7: 不用调试器, 靠 "打印地址 + 看字节" 把问题缩小到一行
2 // 三个现场: 对象切片、悬空引用、容器扩容后旧引用失效
3 #include <cstdio>
4 #include <cstring>
5 #include <memory>
6 #include <string>
7 #include <vector>
8
9 static void hexdump(const void *data, std::size_t bytes, const char *label) {
10     const unsigned char *p = static_cast<const unsigned char *>(data);
11     printf("%-16s @%p : ", label, data);
12     for (std::size_t i = 0; i < bytes; ++i) printf("%02x ", p[i]);
13     printf("\n");
14 }
15
16 // ----- 现场 1: 对象切片 (不崩, 但行为悄悄变了) -----
17 struct Base {
18     int id;
19     explicit Base(int i) : id(i) {}
20     virtual const char *who() const { return "Base"; }
21     virtual ~Base() = default;
22 };
23
24 struct Derived : Base {
25     char extra; // 子类新增的成员
26     explicit Derived(int i) : Base(i), extra('D') {}
27     const char *who() const override { return "Derived"; }
28 };
29
30 static void byValue(Base b) { // ← 按值传: 只复制 Base 那部分
31     printf(" byValue 收到的对象: sizeof=%zu vptr=%p who()=%s\n",
32           sizeof(b), *reinterpret_cast<void **>(&b), b.who());
33 }
34
35 static void byRef(Base &b) {
36     printf(" byRef 收到的对象: sizeof=%zu vptr=%p who()=%s\n",
37           sizeof(b), *reinterpret_cast<void **>(&b), b.who());
38 }
39
40 int main() {
41     printf("=== 现场 1: 对象切片 ===\n");
42     Derived d(7);
43     printf(" 原始对象: sizeof=%zu vptr=%p\n", sizeof(d), *reinterpret_cast<void **>(&d));
44     hexdump(&d, sizeof(d), "Derived 原始");
45     byRef(d); // 引用: 对象没被复制, 虚调用仍然正确
46     byValue(d); // 值传递: Derived 被 "切成" Base, extra 丢了、虚表也换成 Base 的
47     printf(" → 定位手法: 打印 vptr。两个 vptr 不同, 就说明对象被切片了。\\n");
48     printf(" → 修法: 接口一律按引用/指针传递, 或显式 clone 成 unique_ptr。\\n\\n");
49
50     printf("=== 现场 2: 悬空引用 ===\n");
51     const std::string *dangling = nullptr;
52     {
53         std::string local = "临时字符串";
54         dangling = &local;
55         printf(" 作用域内 local @%p 内容=\"%s\\n", (const void *)&local, local.c_str());
56     }
57     printf(" local 已销毁, 但我们还拿着地址 %p\\n", (const void *)dangling);
58     printf(" → 继续解引用就是 UB: 内容可能看起来是对的, 也可能乱码, 随机性正是它可怕的地方\\n");
59     printf(" → 修法: 用 shared_ptr/unique_ptr 表达所有权, 或复制一份数据\\n\\n");
60
61     printf("=== 现场 3: 容器扩容后旧引用失效 ===\n");
62     std::vector<std::string> names{"a", "b"};
63     const std::string &first = names.front();
64     printf(" 扩容前 names.data()=%p, first 引用地址 %p\\n",
65           (const void *)names.data(), (const void *)&first);
66     for (int i = 0; i < 20; ++i) names.push_back("x");

```

```
67     printf(" 扩容后 names.data()=%p, capacity=%zu\n",
68           (const void *)names.data(), names.capacity());
69     printf("  → 数据已经搬家, first 指向的是旧地址 (悬空) \n");
70     printf("  → 修法: 要么先 reserve 足够容量, 要么用下标/迭代器重新取\n\n");
71
72     printf("=== 排坑四步法 (把大问题切成一行) ===\n");
73     printf("  1. 复现: 写出 20 行以内的最小例子, 稳定触发\n");
74     printf("  2. 观测: 打印关键地址、sizeof、vptr、size/capacity、引用计数\n");
75     printf("  3. 对比: 正常路径 vs 异常路径的字节差异 (hexdump 对照) \n");
76     printf("  4. 验证: 用 ASan/UBSan 复跑, 确认修完之后不再报错\n");
77     return 0;
78 }
```